Customer Challenges

Provisioning
Complex
to integrate

How to choose, setup and integrate an operational database, including data synchronization?

How to make it easy for application teams to use?

Scaling
Difficult
to manage

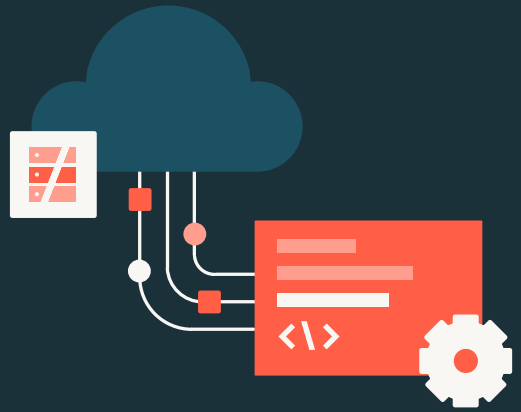
How to scale and troubleshoot custom deployments and data synchronization?

Automation
Outdated developer
experience

How to scale and deploy quickly? How to minimize risk of having multiple databases across dev, test, staging, prod environments?



Applications have evolved



Applications have become stateless, serverless, and intelligent

Databases haven’t

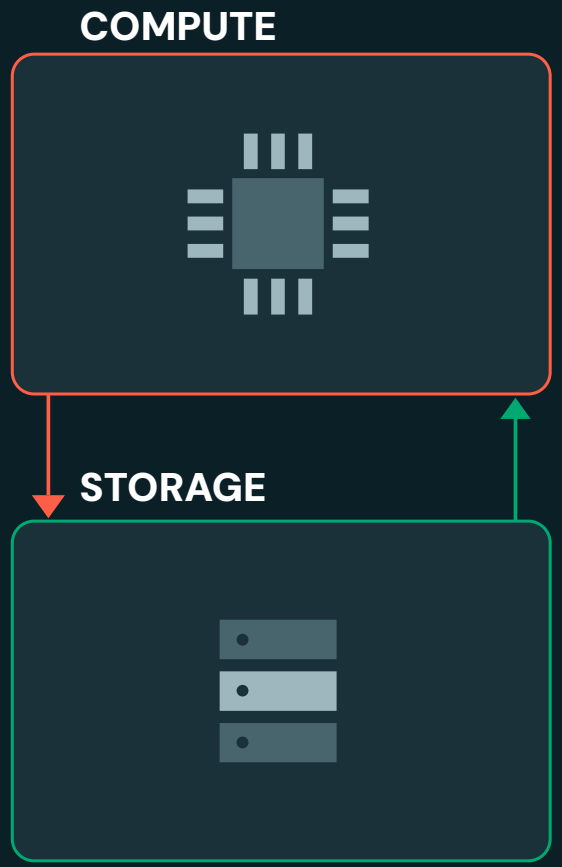


Databases remain monolithic and impossible to move

Lakebase

Fully-managed, serverless Postgres

Separates compute and storage



Lakebase



Great Developer Experience

Branching
& snapshots



Lower TCO

Autoscaling, elasticity, serverless



Integrated Platform

Simplify your stack, with centralized governance and no ETL

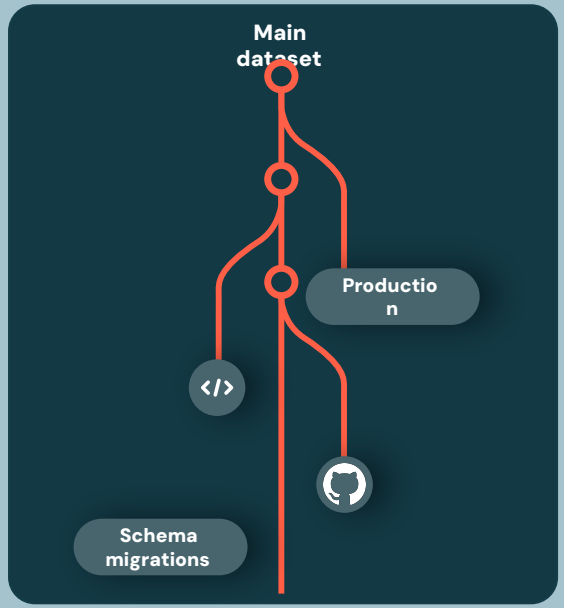
Branching and development

Treat your database like code

Lakebase Branching
Instant, zero-copy isolation lets teams safely build and test without touching production and live workloads.

Git-style workflow for data
Bring CI/CD to your database for feature branches and rapid experimentation.

Ephemeral environments
Create and decommission dev/test branches quickly with no storage duplication.



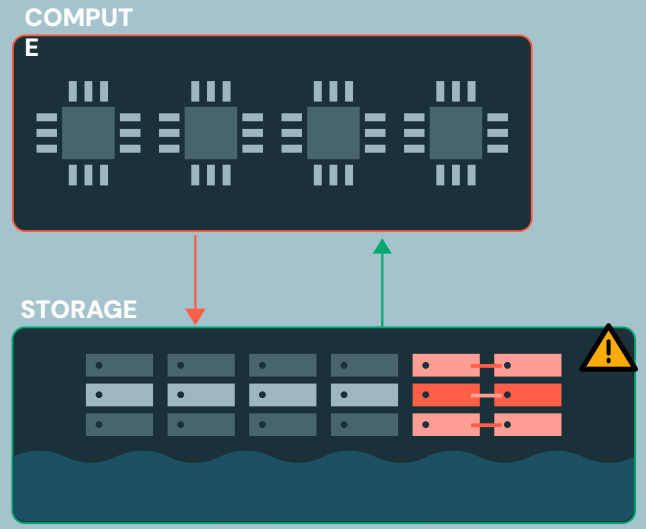
Recovery and backups

Precise restoration, at scale

Scheduled snapshots
Create instant database backups on a schedule.

Rapid, precise recovery
Restore to an exact moment within a managed window (up to 35 days) for quick incident recovery.

Safe validation
Use an isolated, zero-copy branch before applying changes to reduce the risk of bad restores to production.



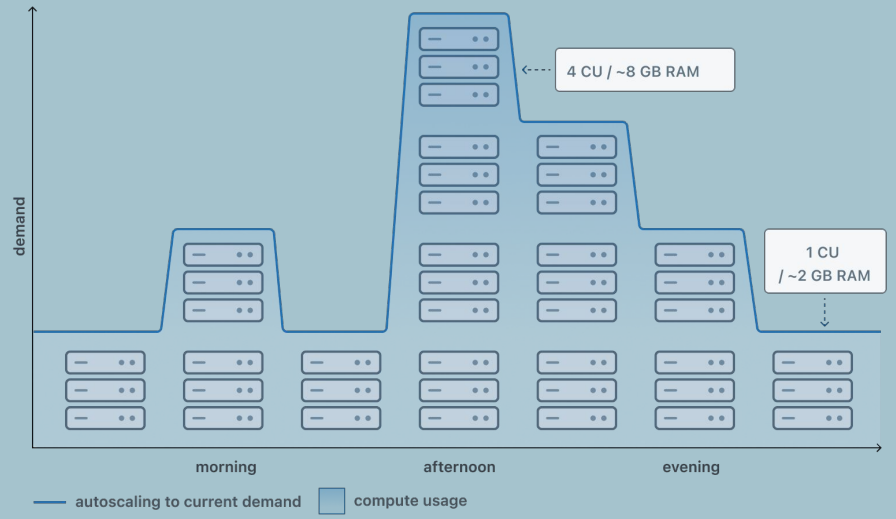
Autoscaling

Fast performance and smart savings, with no manual tuning

Dynamic compute sizing
Compute automatically scales based on workload demand, for high performance even during spikes.

Configurable
Define minimum and maximum compute size to keep costs under control.

No downtime
Scaling operations do not interrupt operations.



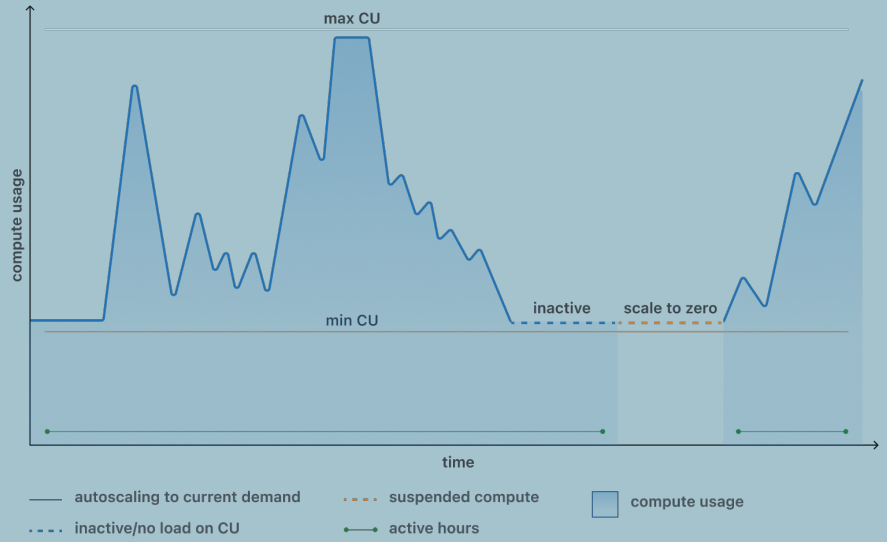
Fast startup and scale to zero

Ready when you need it, without paying for idle capacity

Only pay for active use
Pay only when your database is active. Idle instances drop to zero to keep costs low.

Automatic scale to zero
Unused databases automatically suspend after a configurable idle period with no manual intervention.

Instant readiness
Sub-second start times mean responsive apps and fast setup for dev/testing.



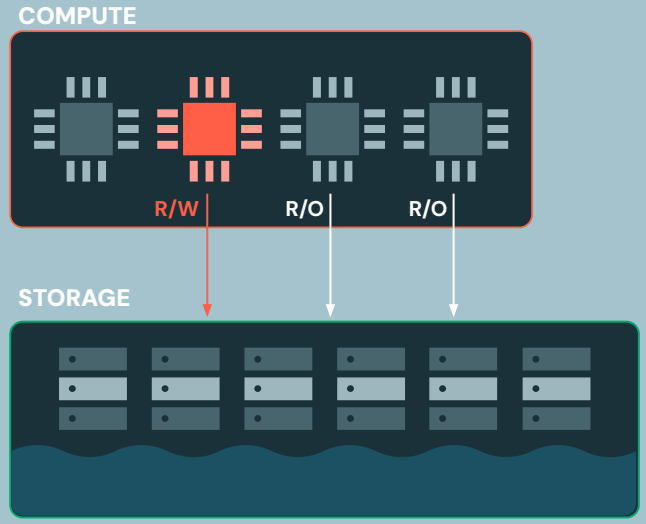
Horizontal read scalability

High concurrency without operational complexity

Read isolation
Dedicated read-only compute serves queries without impacting write workloads.

Independent scaling
Add or remove read query capacity without changing write compute.

Shared data layer
All compute accesses a single underlying data set, with no data duplication.



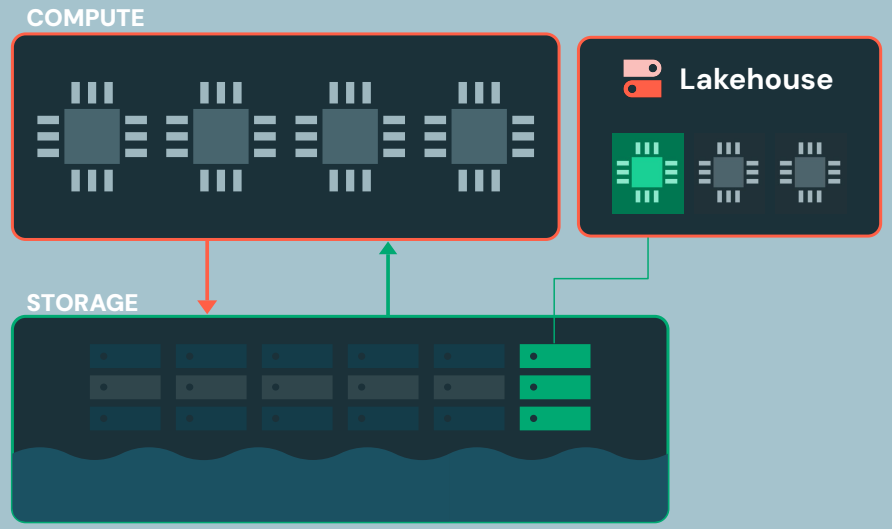
Built into the Data Intelligence Platform

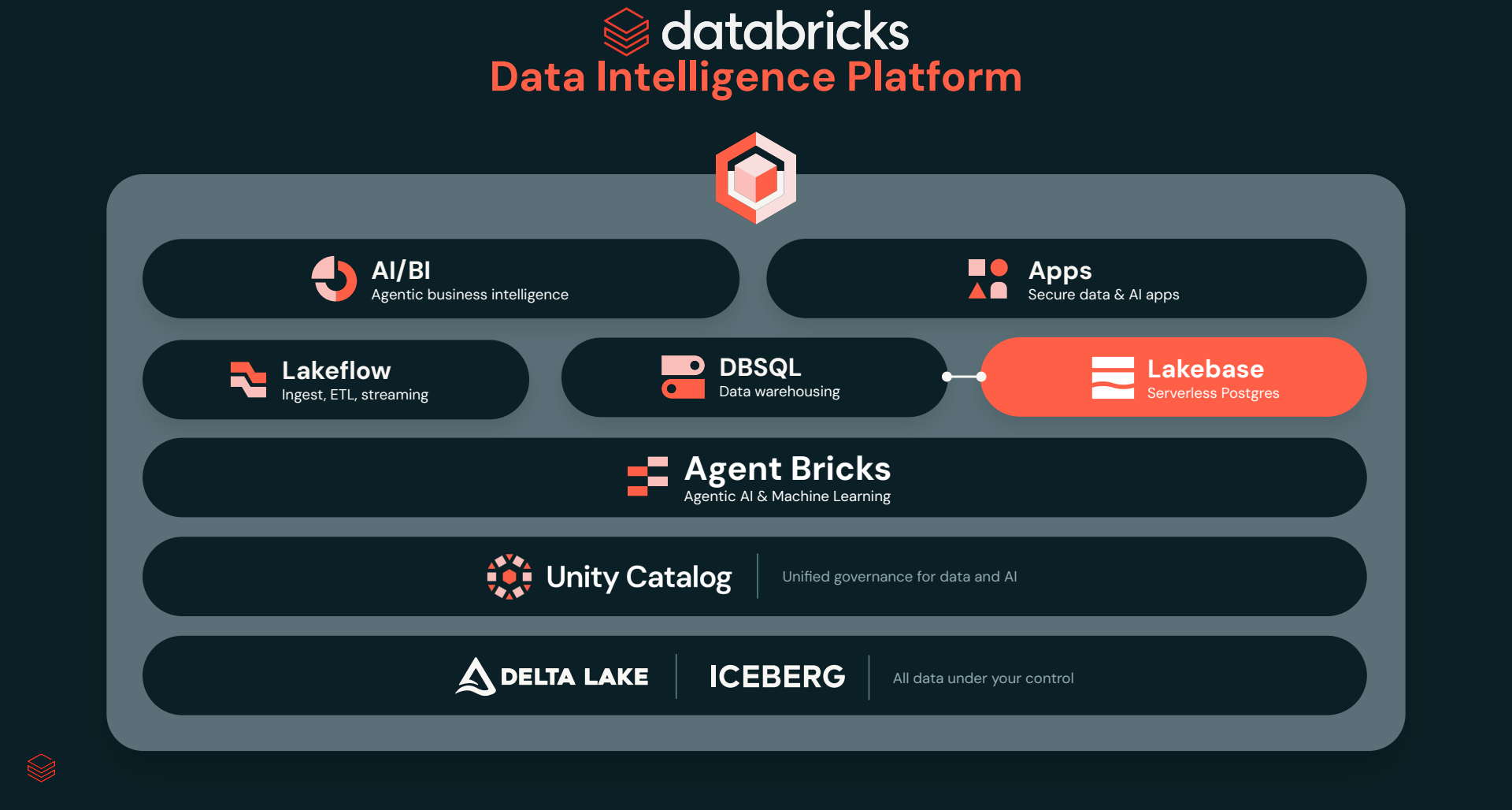
Avoid separate stacks and proprietary governance models

First-class platform compute
Execute operational workloads on the same platform as analytics and AI.

Share a common data foundation
Operational data is immediately available without copying or ETL pipelines.

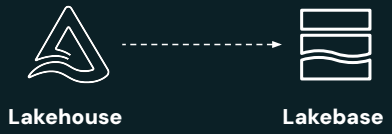
Built-in governance and security
Apply consistent access control, auditing, and policies across all data.




databricks
Data Intelligence Platform
AI/BI
Agentic business intelligence
Apps
Secure data & AI apps
Lakeflow
Ingest, ETL, streaming
DBSQL
Data warehousing
Lakebase
Serverless Postgres
Agent Bricks
Agentic AI & Machine Learning
Unity Catalog
Unified governance for data and AI
DELTA LAKE
ICEBERG
All data under your control

What can you do with Lakebase?

Serve low latency, high concurrency



Personalized recommendations

Customer segmentation

Feature store

Build AI and traditional apps



Order processing

Interactive workflow sign-off

State for an agent

Analyze data in the lakehouse



Order history (analytics)

Chatbot history (training data)

What should you NOT do with Lakebase?

- Large scale migrations. For customers considering PostgreSQL migration → use Lakegate to assess readiness before committing (16 automated checks, readiness score, phased migration playbook)
- Migration from non-postgres databases (Especially true for GraphQL and NoSQL DBs!)
- Run workloads which require more than:
  - 4000 concurrent connections
  - 16TB of Storage
- Run OLAP workloads in an OLTP database, expecting improved performances (it won't)
- Splitting a large dataset into multiple branches or projects to workaround those restrictions is **strongly** discouraged

©2025 Databricks Inc. — All rights reserved



Qualification: Is Lakebase good fit?

Use this to quickly validate fit for Lakebase

	✅ Green Flags (Strong Fit)	⚠️ Yellow Flags (Proceed with Caution)	🛑 Red Flags (Walk Away)
Workload Type	**New Apps & Portals:** Internal tools, customer-facing apps, or operational dashboards . **AI Agent State:** Storage for chatbot session history (long/short-term memory) . **Reverse ETL:** Need to serve curated Lakehouse data (Gold tables) to apps without complex pipelines.	**Postgres Migrations:** Moving from RDS/Aurora is okay if they need autoscaling/branching, you should win your first proof-point before going after a migration	**Large-Scale Migrations:** Migrating massive legacy Oracle/SQL Server estates . **Heavy Analytics (OLAP):** Running warehouse-style reporting directly on the transactional DB (use DBSQL instead).
Data & Scale	**Variable Traffic:** "Spiky" workloads or dev/test environments that sit idle (perfect for Scale-to-Zero) . **Standard OLTP:** Needs <10ms latency for point lookups.	**Size > 16TB:** Dataset exceeds the 8TB limit for Autoscaling, understand their current design as well as our storage increase timelines..	**High-Write NoSQL:** Extremely high-throughput write workloads better suited for DynamoDB/Cassandra.
Team & Ops	**Fast Cycles:** Teams demanding rapid CI/CD, ephemeral environments, or feature branching . **Data + App Convergence:** Data teams owning the "last mile" of serving.	**Strict Silos:** App teams with zero interest in the Data Platform/Lakehouse ecosystem.	**Specialized Tech:** Requirement for proprietary technology that doesn't integrate with Lakebase (qualify their ecosystem early)



Postgres database engine

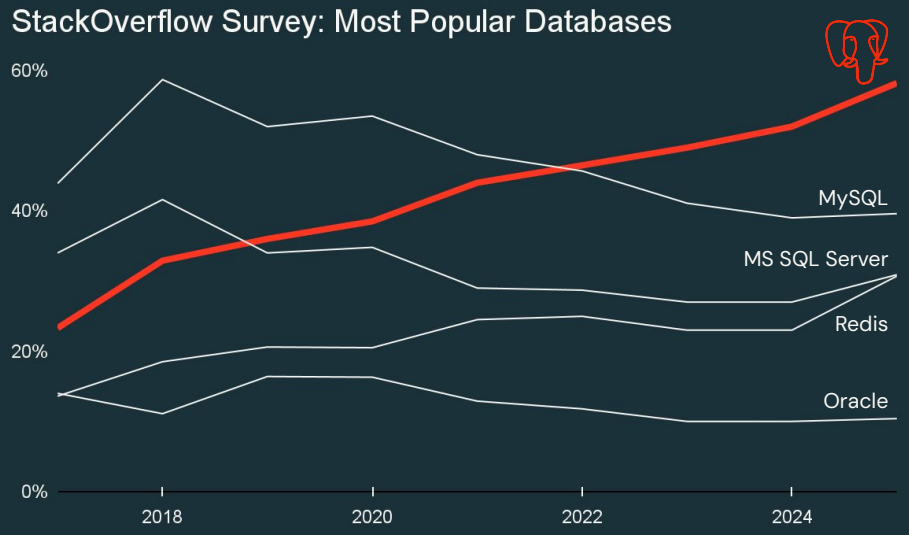
Fastest growing database engine of last decade

Postgres Engine

High perf: <10ms latency, >10ks QPS	✓
Full Postgres SQL engine: transactions (ACID), indexes, views, pk/fk enforcement, joins, CTEs, triggers, stored procedures	✓

Rich ecosystem

Extensions: incl: PostGIS, pgvector	✓
Drivers: Java, Python, etc	✓
Tools: pgAdmin, DBeaver, PSQL	✓



Separation of storage and compute

Fully managed, easy to use, and enterprise ready



Decoupled compute and storage

Independent compute/storage scaling	✓
Pause/resume compute	✓
Databricks Serverless Infra: hardened compute & encrypted storage	✓

Secure

Databricks Identity & OAuth	✓
PrivateLink / IP ACLs	✓
Transport Layer Security (TLS) encryption	✓

High-availability and recovery

Multi High Availability Zone Failover	✓
Readable Secondaries	✓
Point-in-Time Recovery up to 35 days powered by copy-on-write clone	✓

Management / observability

Built in monitoring	✓
Databricks Apps resource	✓
Terraform; DAB; Billing Tags	✓

Integrated with Lakehouse

Easy to use



Fully-Managed Data Synchronization

Sync from Lakehouse table - Postgres	✓
Sync from Postgres - Lakehouse table	✓
Snapshot, Triggered, Continuous	✓
Multiple table syncs in a pipeline	✓

Integrated with Unity Catalog

Register Postgres database as Catalog	✓
Governed by UC	✓
Lakehouse Federation	✓

AI Platform: Online Feature Store

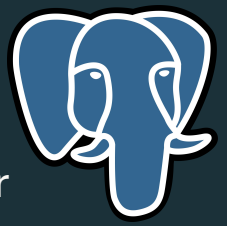
High performance feature lookup	✓
Feature store for real-time serving, with offline training data in Lakehouse	✓

Query Editor and Schema Browser

Native Postgres query editor	✓
Browse Postgres schema	✓



Postgres Extensions



- We support an ever growing list of extensions, which are documented on our website
- The list of available extensions can also be retrieved from Postgres directly using `SELECT * FROM pg_available_extensions ORDER BY name;`
- Installing an extension is as easy as `CREATE EXTENSION <extension_name>;`
- Amongst the popular extensions: pg_stat_statements (query metrics), pgcrypto, pg_prewarm (cache prewarming), neon, postgis (geospatial), pg_graphql (GraphQL support), pg_vector

OBS: We cannot support extensions that directly manipulate Postgres storage as this would be incompatible with our storage infrastructure



Pricing

Learn more → public pricing page

Stop paying for idle time: True serverless pricing that scales to zero.

1. The Model: Two Simple Meters

- **Compute (DBUs):** Charged per second.
  - **Autoscaling:** Pay only when the database is active. $0 when idle.
- **Storage:** Standard monthly storage rates.

2. The Launch Promotion (Limited Time)

- **50% OFF Compute:** We are applying a 50% discount on all Lakebase Compute DBUs.
- Validity: Active now through **January 31, 2027.**

3. Key Dates for Billing

- **February 1, 2026:** Billing begins for Autoscaling instances (with 50% discount applied).

Note: Full pricing details and FAQ available at our public pricing page (including FAQ)

Monitoring

Billing - System tables

- Usage by billing_origin_product = DATABASE (replaces ONLINE_TABLES)
- **Compute: PREMIUM_DATABASE_SERVERLESS_COMPUTE_<REGION> SKU, billed in DBUs**
- **Storage: PREMIUM_DATABRICKS_STORAGE_<REGION> SKU, billed in DSUs (GB-month)**
- **Synced tables (Delta → Lakebase): Uses Lakeflow Spark Declarative Pipelines (dlt_pipeline_id in usage_metadata)**
- **usage_metadata.database_instance_id identifies the specific Lakebase instance**
- Cost analysis via system.billing.list_prices for time-banded rate lookups

Audit

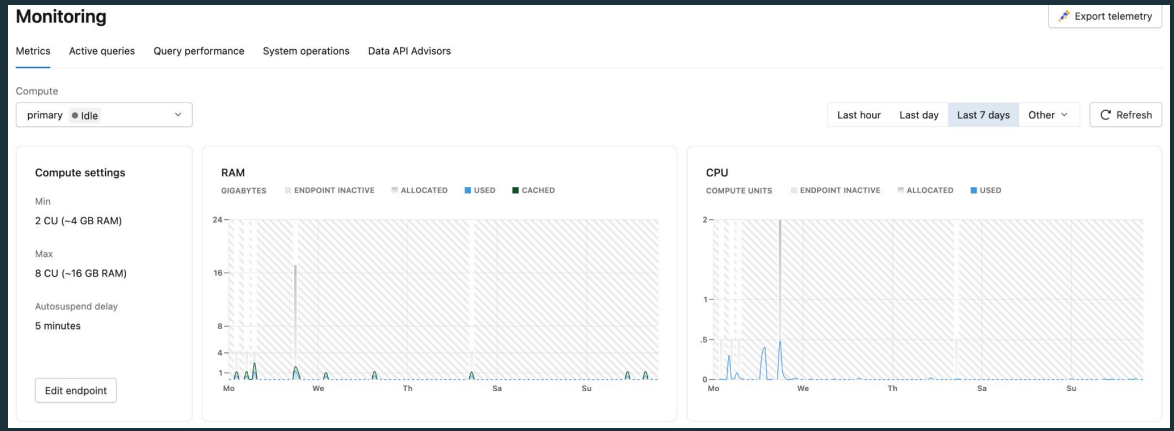
- Lakebase access/creation/deletion available at system.access.audit
- DBSQL access to Lakebase tables available in system.query.history
- Data API (PostgREST) is now GA with full CRUD, OAuth auth, and RLS – dedicated Data API audit logs not yet confirmed in system tables

©2025 Databricks Inc. — All rights reserved

Built-in Monitoring (Lakebase UI)

The **database project metric tab** provides a dashboard for monitoring the database usage

- Memory and CPU utilization
- Transactions per second - *general observability for committed transaction throughput.*
- Rows per second - *rows fetched, returned, inserted, updated, and deleted as tracked by the PostgreSQL database*
- Open connections
- Storage utilization
- CPU utilisation %
- Page read throughput %
- Buffer cache hit rate %
- Local SSD cache hit rate %
- Deadlock per second

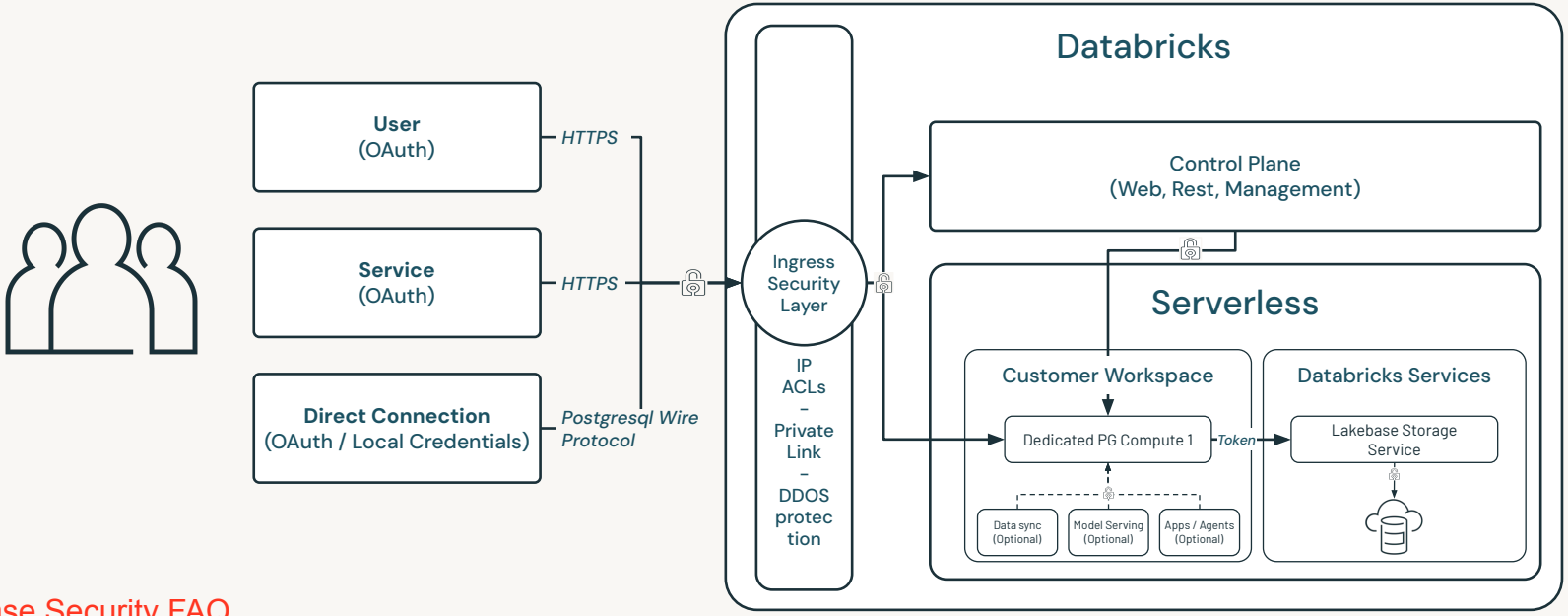


Additionally, it's possible to use tools like pgAdmin or other pg-integrated tools to build dashboards to monitor the health of your postgres instance

©2025 Databricks Inc. — All rights reserved

Lakebase Connectivity

Customers can connect to Lakebase using Databricks native services (notebooks, SQL Editor, REST API) and directly to the PostgreSQL service.



Lakebase Security FAQ

The Serverless Security Overview: Lakebase + Lakebase: Security Video

©2025 Databricks Inc. — All rights reserved

Lakebase Authentication

Lakebase provides PostgreSQL native and Workspace authentication options.

PostgreSQL Authentication



PostgreSQL Lakebase account authentication

- Workspace user credentials (OAuth)
- PostgreSQL login

Databricks managed Authentication



Databricks Service Authentication

- Workspace user authentication (OAuth)
- DB managed OAuth M2M authentication

©2025 Databricks Inc. — All rights reserved

Lakebase Authorization

Lakebase provides PostgreSQL native and Unity Catalog authorization options.

PostgreSQL Authorization



PostgreSQL Lakebase account authorization

- Native permissions and privileges

Unity Catalog Governance



Unified Authorization

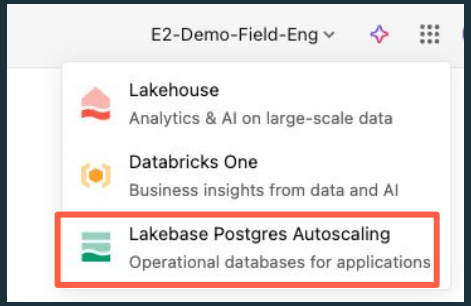
- UC-managed governance
- Native services such as Notebooks, REST API, DB SQL, PostgREST API, Model Serving and Apps

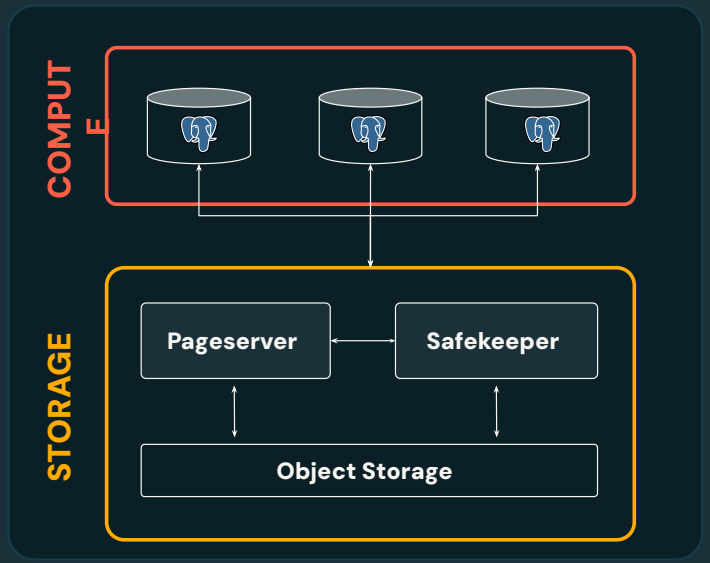
©2025 Databricks Inc. — All rights reserved

Lakebase Autoscaling

What's Included:

- Disaggregated storage and compute
- Autoscaling
- Scale to zero (5 min default)
- Branching
- Point-In-Time-Recovery (PITR)
- History retention
- Customer-Managed Keys (CMK)





©2025 Databricks Inc. — All rights reserved

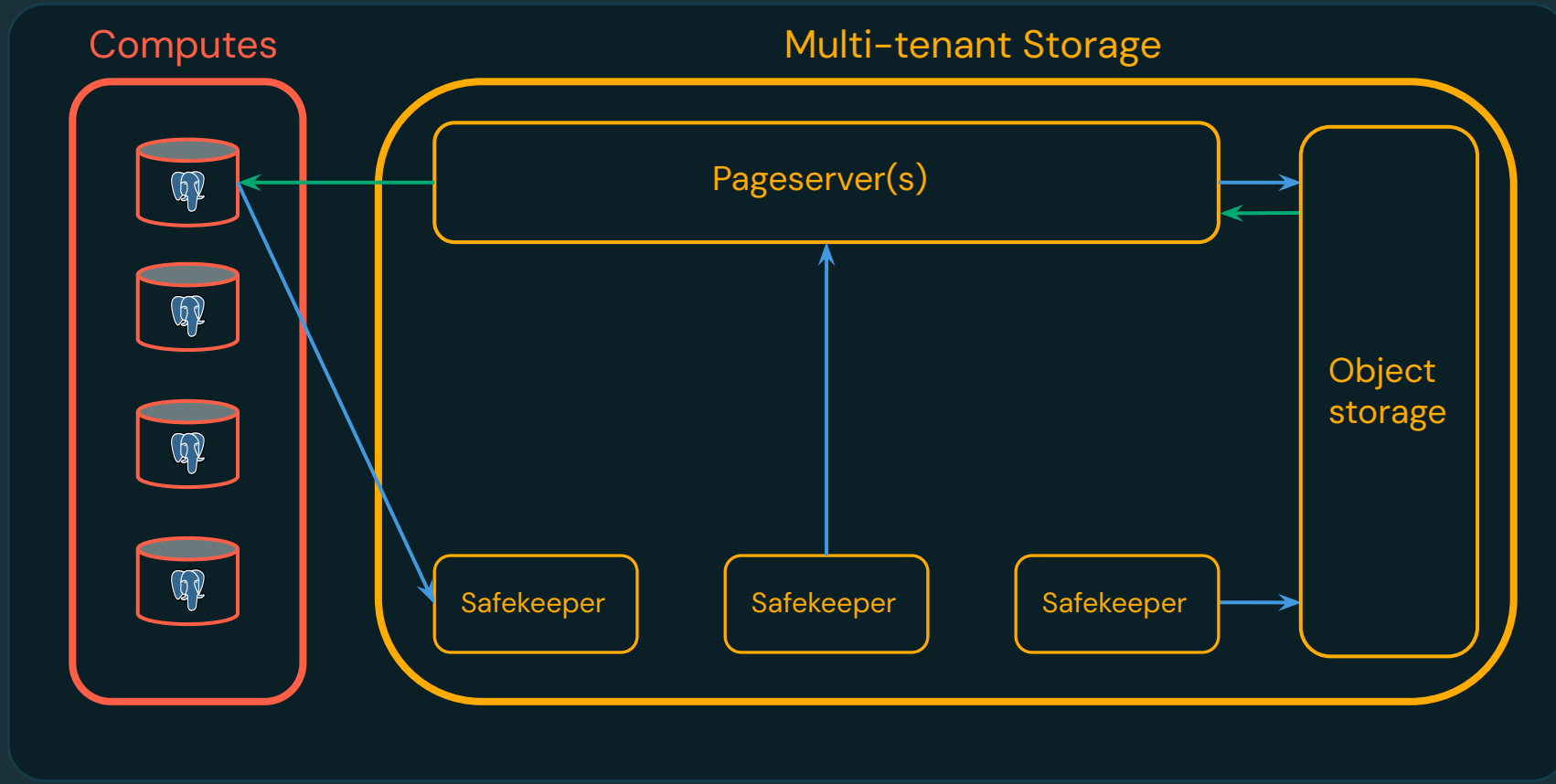
Why Autoscaling?

Limitations of Traditional Monolithic PostgreSQL

- **Single point of failure results in downtime or data loss**
  - Machine or datacenter failure
  - Disk failure
- **Scaling and Performance Limits**
  - Inefficient resource utilization across storage/compute
  - Inefficient resource utilization across time (peak times)
- **Recovery and Resiliency**
  - Slow restores from backups (dependent on storage size)
  - HA/DR complex to set up and maintain
- **High operational overhead**
  - Capacity planning, backups, replication, failover, manual recovery, etc.

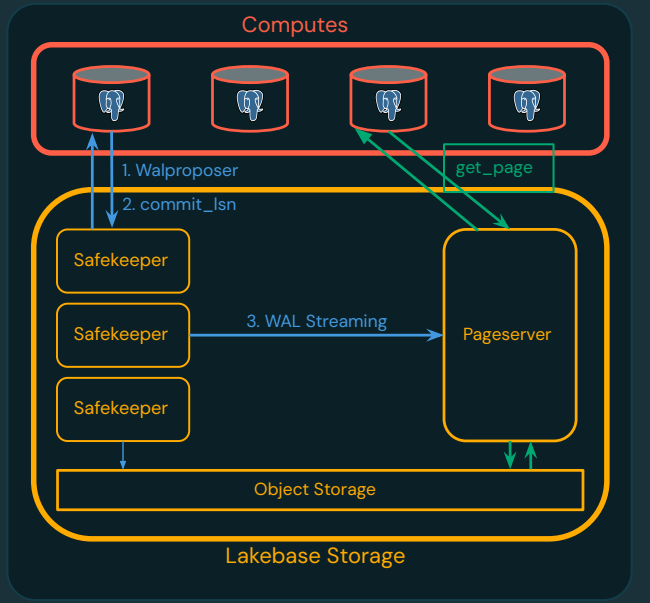


Lakebase Architecture Overview



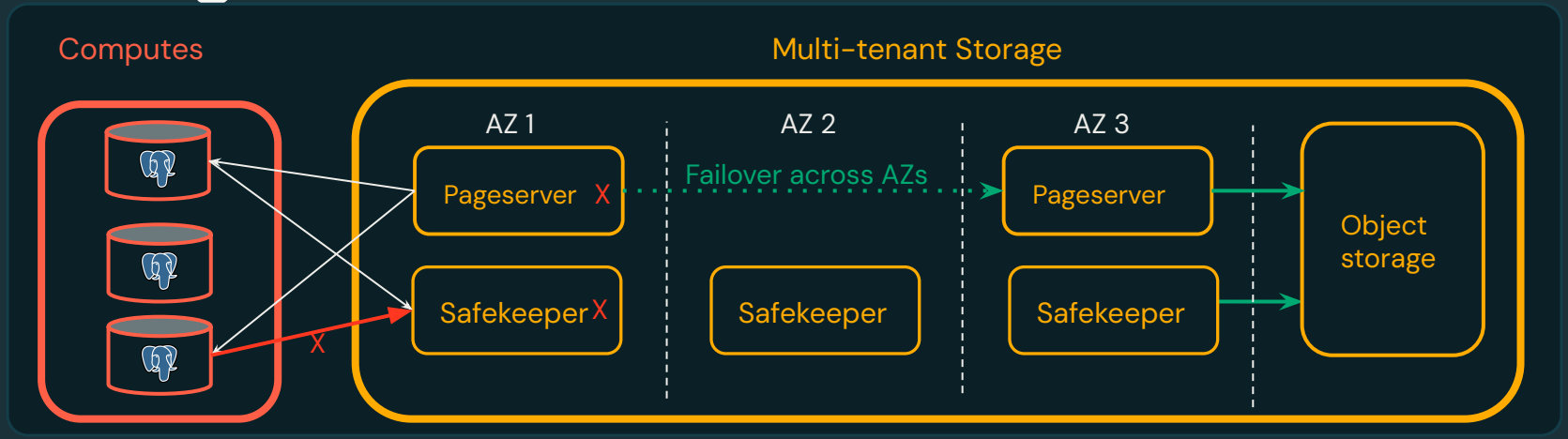
Storage

- The customer dataset resides on one or more Pageserver(s) and on Object Storage. Data are not stored locally*
- The Lakebase storage layer is multi-tenant and cannot be hosted in the customer workspace
- **Writes:** Compute writes to 3 safekeepers. When the safekeepers reach consensus, the writes are persisted to object store and streamed to pageserver. When a write is persisted, it is considered committed
- **Reads:** Compute reads from Pageserver. If the requested page is not available locally, it is reconstructed from Object storage.



* exception made for Postgres temp files and WAL files that needs to be streamed

Storage HA Overview



- Each Postgres instance uses 3 safekeepers located in different AZs. If one Safekeeper or AZ fails, the system can still reach consensus. Built-in redundancy. Immediate recovery.
- Each Postgres instance is hosted on two pageservers: one active node and one secondary failover node, both nodes being located in different AZs. If a Pageserver fails, reads are automatically routed to the secondary failover node. Recovery in a few seconds.
- Object storage has built-in multi-AZ redundancy. Immediate recovery.